PIC 10A 1A

TA: Bumsu Kim

This content is protected and may not be shared, uploaded, or distributed.

The author does not grant permission for these notes to be posted anywhere without prior consent.

UCLA

Today…

- Pointers
- HW 8 Hints

Pointers

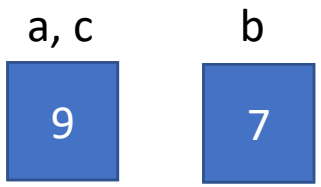
Pointers

- Recall that a reference to a variable is just “another name” of the other

```
class T {
        // ...
};
T tvar;                 // variable of type T
T& ref_tvar = tvar;     // a reference to a type T variable
```

- And they both refer to the same “address” in the memory

```
int a = 5;
int b = a;
int& c = a;
cout << a << ' ' << b << ' ' << c << endl;
b = 7;
cout << a << ' ' << b << ' ' << c << endl;
c = 9;
cout << a << ' ' << b << ' ' << c << endl;
```



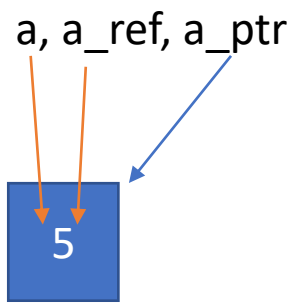
UCLA

Pointers

- So, a reference, in fact, refers to the value in some address
- Pointers are used to “point” to those address

```
int a = 5;
int& a_ref = a;
int* a_ptr = &a; // address of a
cout << a << ' ' << a_ref << ' ' << *a_ptr << endl;
a_ref = 7;
cout << a << ' ' << a_ref << ' ' << *a_ptr << endl;
*a_ptr = 9;
cout << a << ' ' << a_ref << ' ' << *a_ptr << endl;
```

```
5 5 5
7 7 7
9 9 9
```



- Note that, they store the “address” so you can’t do something like

```
a_ptr = a;
```

a value of type "int" cannot be assigned to an entity of type "int *"

Search Online

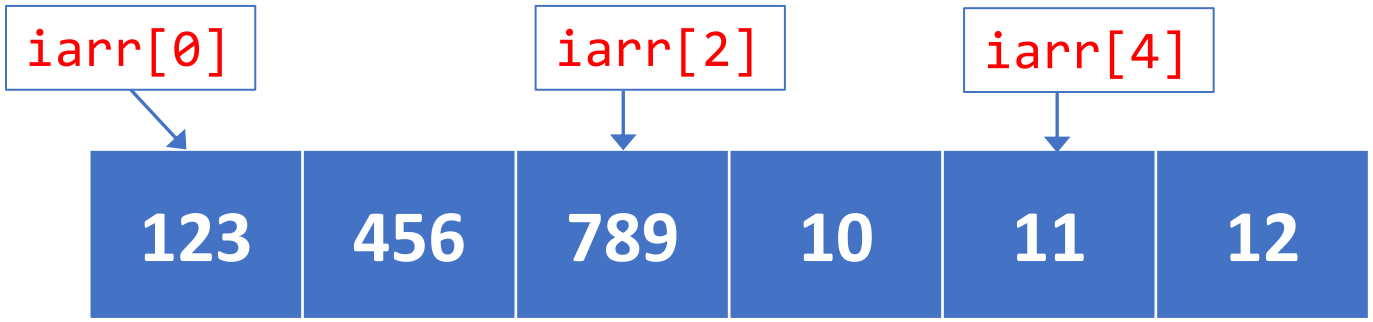
UCLA

Pointer Arithmetic

- To access the value stored in the address where your pointer points, use the ***dereference operator*** (*)

```
int a = 5;
int* a_ptr = &a; // address of a
*a_ptr = 9;   // dereference to access the value
cout << a << ' ' << *a_ptr << endl; // again, dereference to access the value
```

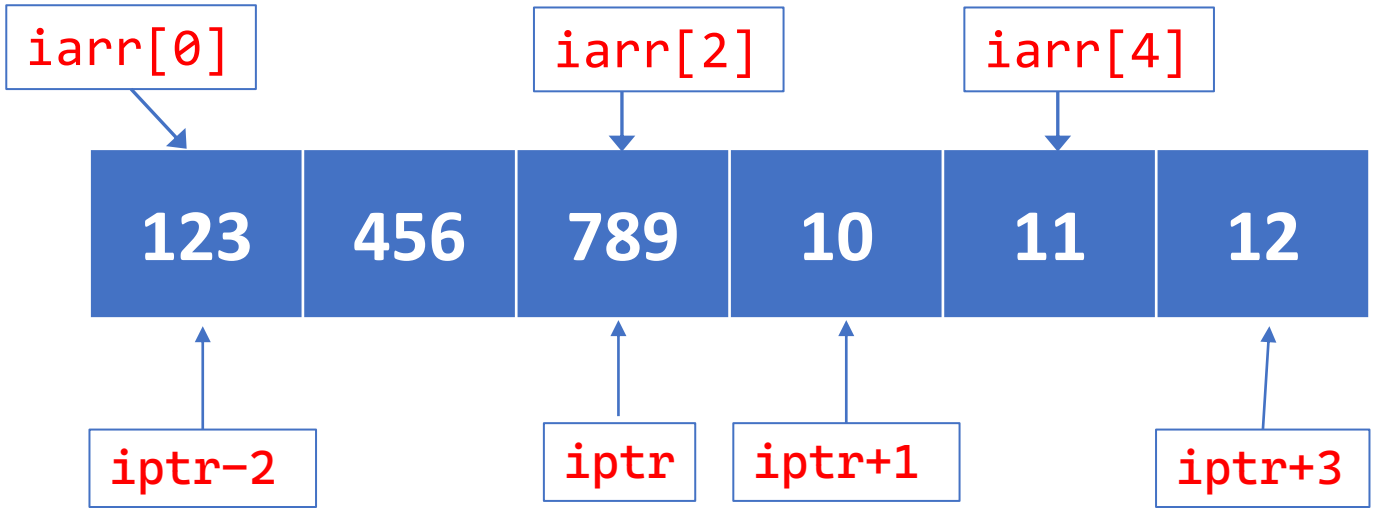
- You can also "move around" the address using "pointer arithmetic"
- Let's say you have an `int` array of size 6:



UCLA

Pointer Arithmetic

- An `int` array of size 6 may look like (in the memory):




```
int* iptr = &(iarr[2]);
// &(2nd elt of iarr)
```

- Let `iptr` be a pointer pointing to the 2nd element of the array, then you can move around by adding/subtracting numbers from its address
- This is called "pointer arithmetic" (just like "integer arithmetic" but no *,/,%, etc.)
  - +, -, and ++, -- are allowed
  - Additionally, comparisons (==, <, >) are also available

Arrays vs Pointers

```
size of a: 20
size of aptr: 8
```

- Arrays "decay" to pointers if it needs to be
- But still, they are DIFFERENT!

```
int a[] = { 1,2,3,4,5 };
int* aptr = a;
cout << "size of a: " << sizeof(a) << endl
<< "size of aptr: " << sizeof(aptr) << endl;
```

- You can compute the length of the array using "`sizeof(a)/sizeof(int)`"
- But then why do you still need to pass the array size to a function? (recall HW 5, for example)

Exercise 1: Equals

(This is essentially Exercise E6.8 of the book)
Write a program equals.cpp that implements the function

```
bool equals(int a[], int a_size, int b[], int b_size)
```

UCLA

Arrays vs Pointers

- It is because an array ***decays*** to a pointer when it is passed to a function
- You can never pass an array (by value), nor return an array
  - Passing arrays is not an error, but it decays to a pointer
  - Returning an array is an error!

- *Remark*: In fact, arrays can be passed by reference

```
int f(int(&a)[3]) {
    cout << "*size of this array: " << sizeof(a) << endl;
    return 0;
}
```

  - This function can get an array of size 3, and the output is

```
*size of this array: 12
```

UCLA

Arrays vs Pointers

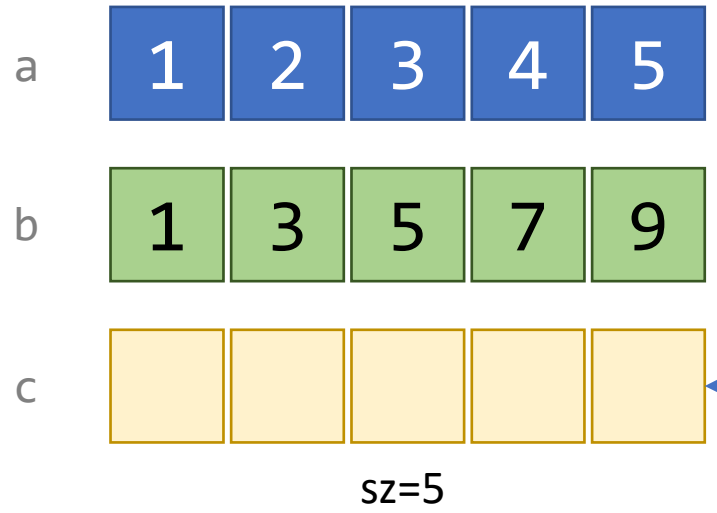
- Since the implicit conversion [array → pointer] is allowed, you can use arrays in every expression where pointers are expected

```
void add_arrays(int a[], int b[], size_t sz, int *c) {
    for (size_t i = 0; i < sz; ++i) {
        *(c++) = *(a++) + *(b++);
    }
}
```

- How does the function work?

UCLA

Arrays vs Pointers




```
void add_arrays(int a[], int b[], size_t sz, int *c) {
    for (size_t i = 0; i < sz; ++i) {
        *(c++) = *(a++) + *(b++);
    }
}
```

At least "sz" places must be reserved!
(otherwise it may corrupt your data and result in undefined behavior)

- Pointers move forward by ++
- The post-increment operator returns the original value
- Access the value stored in an address via the dereference operator *

UCLA

HW 8 Hints

HW 8 Hints

- They both have pretty short answers
- Consider the following loop structure

```
for (int i = 0; i < size; ++i, ++arr) {
    if (/* some condition */) {
        /* Do something */
    }
}
```

- The 99% of the homework is to figure out what the condition should be, and what to do in the if clause!

UCLA

Next time,

- We will have review slides about everything we covered in this course!

UCLA